

Actividad final integradora: El asistente de preguntas frecuentes de Nube Sur

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · Introducción a modelos generativos y consumo por API

MOMENTO

Al cierre del módulo

MODALIDAD

Individual, con entrega en el LMS

CALIFICACIÓN

Pauta de 40 puntos, exigencia 60 %

COMPETENCIA DEL MÓDULO

Implementar la arquitectura básica de una aplicación que integre modelos de IA mediante APIs y preprocesamiento de texto para consumo por modelos de lenguaje, garantizando el consumo eficiente y la evaluación de resultados en un entorno de desarrollo de software

CONTENIDOS DEL PLAN

Los cuatro contenidos del módulo, todos integrados: Modelos generativos y ecosistema tecnológico · Integración y consumo de APIs · Diseño de prompts para APIs generativas · Pipeline de preprocesamiento y evaluación básica de texto

Contexto

Nube Sur (empresa ficticia) vende software de gestión a pymes. Su mesa de ayuda atiende a los clientes por correo, formulario web y chat. Durante el módulo construiste piezas para ella: la arquitectura de su plataforma de soporte, un cliente que conversa con los modelos, prompts que clasifican tickets y un pipeline que limpia textos y mide la calidad de lo que el modelo genera.

Ahora el jefe de soporte trae un problema nuevo. En una semana de muestra contó que 4 de cada 10 consultas del chat preguntaban algo que ya está respondido en la lista de preguntas frecuentes de la empresa. Los agentes copian y pegan la misma respuesta varias veces al día, y a veces la adaptan de memoria, con datos que ya no están vigentes: un plazo antiguo o un menú que cambió de nombre.

Quiere un asistente que conteste esas consultas con la respuesta publicada y que derive a un ejecutivo todo lo demás. Pone tres condiciones: que **nunca invente** (un precio, un plazo o una función que no existe), que **no envíe datos personales** de los clientes a un servicio externo y que **el gasto en la API se pueda medir y controlar**.

El desafío

Construye y evalúa, de forma individual, la primera versión del asistente de preguntas frecuentes de Nube Sur. Dada la pregunta de un cliente, el asistente:

1. la limpia y la normaliza;

2. busca la pregunta frecuente más parecida comparando embeddings;
3. si la similitud no alcanza un umbral que tú defines, responde "Te derivo con un ejecutivo." sin llamar al modelo de generación;
4. si lo alcanza, le pide al modelo una respuesta breve basada **solo** en la respuesta publicada de esa pregunta frecuente, en formato JSON;
5. registra la calidad de sus respuestas y los tokens que gasta.

Al final, le recomiendas al jefe de soporte, con números, si el asistente pasa a un piloto en el chat, pasa con ajustes o no pasa.

Lo que vas a construir

El trabajo tiene cinco partes. Cada una deja archivos en tu repositorio; la sección Entrega los resume.

Parte 1 · Arquitectura y decisiones

En el `README.md`, sección «Arquitectura»:

1. **¿Por qué IA generativa?** Explica en 5 a 8 líneas para qué sirve aquí la IA generativa y por qué no basta una solución basada en reglas con palabras clave (por ejemplo, "si la pregunta contiene «contraseña», responde F1"). Demuéstralo con Q1 y Q2 de los insumos. Indica también una tarea del asistente que sí conviene resolver con una regla, como rechazar un mensaje vacío o demasiado largo antes de gastar tokens.
2. **Tipos de IA y de modelos.** En una tabla, clasifica los modelos que usa el asistente: el tipo de IA generativa (texto) y la familia de cada modelo (un modelo autorregresivo basado en transformers para redactar; un modelo de embeddings para buscar, típicamente un transformer codificador: verifícalo en la ficha del modelo que uses). Agrega los que descartas, imagen, audio, video y modelos de difusión, con una línea de por qué.
3. **Casos de uso.** Indica qué casos de uso de la IA generativa cubre el asistente (preguntas y respuestas o QA, y reformulación de la respuesta publicada) y uno que podrías agregar después, como la generación de tests para tus funciones o el resumen de la conversación cuando el asistente deriva. Di cómo validarías esa salida.
4. **Diagrama funcional.** Dibuja el diagrama (Mermaid o imagen) con los cinco componentes: cliente (el chat de Nube Sur), servidor API (en tu versión, el módulo `asistente.py`), módulo de preprocesamiento, modelo de IA (generación y embeddings, por API) y vector store (los embeddings de las preguntas frecuentes guardados en un archivo). Numera el flujo de datos de extremo a extremo con una pregunta de ejemplo, explica en una línea la función de cada componente y marca dónde vive la clave. No necesitas publicar un servidor web.
5. **Solicitud–respuesta y pipeline.** Señala qué parte del asistente funciona como solicitud–respuesta (la consulta en vivo) y qué parte como pipeline (preparar e indexar una sola vez las preguntas frecuentes; evaluar por lotes las preguntas de prueba).
6. **Tecnologías.** En una tabla, las tecnologías y librerías que eliges para cada componente y las que descartas en esta versión, con una línea de por qué: API de OpenAI o de Hugging Face, Hugging Face Transformers, PyTorch, TensorFlow, LangChain y Diffusers. Por ejemplo: no

necesitas PyTorch ni TensorFlow porque no entrenas ni ejecutas modelos en tu equipo, y LangChain no se justifica para un flujo de cinco pasos (la orquestación la trata el módulo 3 del plan, «Orquestación de flujos y agentes mediante frameworks»).

- 7. Justificación.** Justifica la arquitectura con tres criterios: escalabilidad (qué pasa si el chat recibe diez veces más consultas), seguridad (la clave vive solo en el servidor, en una variable de entorno, y los datos personales se quitan antes de salir) y mantenimiento (cambiar de proveedor o de modelo con una variable de entorno).

Parte 2 · El cliente de la API

- 1. Reutiliza o amplía** el cliente que construiste en el módulo: `ClienteIA` o el paquete `nubesur_ia` del ABPRO «Un cliente, dos proveedores». Indica en el README qué reutilizaste y qué agregaste.
- 2. Tres operaciones**, con `httpx` o `requests`: `listar_modelos()`, una solicitud GET que al arrancar comprueba que la clave y los modelos configurados existen; `embeddings(textos)`, un POST que devuelve un vector por texto en el mismo orden de entrada; y `generar(mensajes, temperature, max_tokens)`, un POST al endpoint de chat que devuelve el texto y el campo `usage`. Cada operación arma su cuerpo JSON y lee la respuesta solo después de revisar el código de estado.
- 3. Configuración.** Todo sale de variables de entorno: `PROVEEDOR_IA` (`openai` o `huggingface`), `OPENAI_API_KEY` o `HF_TOKEN`, `MODELO_CHAT` y `MODELO_EMBEDDINGS`. El archivo `.env` va en `.gitignore`; el `.env.example` lleva los nombres, sin valores.
- 4. Tiempo de espera** explícito en cada solicitud (con `requests` no hay uno por defecto).
- 5. Errores.** Un 401 produce un mensaje claro que apunta a la clave, sin reintentar. Un 429 o un 5xx se reintenta hasta tres veces con esperas crecientes, respetando el encabezado `Retry-After` si viene; pero un 429 con el código `insufficient_quota` (cuota o crédito agotado) no se reintenta, porque esperar no lo resuelve. Un tiempo agotado o un error de conexión produce un mensaje claro. En todos los casos, el asistente responde "Te derivo con un ejecutivo." y registra el error, sin mostrarle al cliente el detalle técnico.
- 6. Docstrings** en cada clase y método público: qué hace, Args, Returns y Raises.
- 7. Pruebas unitarias.** Al menos cinco, que pasen sin conexión y sin clave real, con la API simulada (`httpx.MockTransport` o `unittest.mock`). Como mínimo: `generar()` hace un POST al endpoint de chat con `Authorization: Bearer` y devuelve el texto y `usage`; `embeddings()` respeta el orden por `index`; `listar_modelos()` usa GET; un 401 produce un error claro con su código; y un 429 seguido de un 200 termina bien tras reintentar (inyecta la función de espera para que la prueba no espere de verdad).
- 8. Tabla de endpoints** en el README: proveedor, operación (listar modelos, generación, embeddings), modelo que usaste, método, URL, autenticación, campos obligatorios del cuerpo, dónde viene el resultado y el enlace a la página de la documentación oficial que consultaste, con su fecha. Las URL y los nombres de modelos cambian: escribe lo que dice la documentación vigente, no lo que recuerdas.

Parte 3 · Preprocesamiento y búsqueda

1. Funciones reutilizables en `preprocesar.py` (amplía la del módulo si la tienes), una por etapa:

- `limpiar(texto)`: decodifica las entidades HTML (`html.unescape: ´`; pasa a á), quita las etiquetas y el ruido (firmas, "Enviado desde mi...", signos repetidos) y reemplaza los datos personales por marcas (`[correo]`, `[enlace]`), sin borrar el contenido de la pregunta;
- `normalizar(texto, sin_tildes=False)`: unifica espacios, comillas y mayúsculas y, solo si se pide, quita las tildes;
- `tokenizar(texto, lematizar=False, sin_stopwords=False)`: tokeniza con spaCy (`es_core_news_sm`) o NLTK y, si se pide, lematiza y quita stopwords;
- `preprocesar(texto, destino="modelo")`: encadena las etapas según el destino. Con `destino="modelo"` (embeddings y generación) devuelve `limpiar(texto)`; con `destino="comparar"` (TF-IDF, CountVectorizer y métricas) devuelve los tokens de `normalizar(limpiar(texto), sin_tildes=True)`. La búsqueda y la evaluación llaman solo a esta función, con el destino que corresponde.

2. Estructura modular. Explica en el README el orden de las etapas y qué versión del texto usa cada componente: los embeddings y el modelo de generación reciben el texto limpio y natural, con mayúsculas y tildes; TF-IDF, CountVectorizer y las métricas, el texto normalizado y tokenizado.

3. Conceptos, con Q6. Muestra los tokens de Q6 limpia, sus lemas, cómo queda sin stopwords y sus bigramas. Revisa si tu lista de stopwords incluye "no" y decide si conviene quitarla en la búsqueda. Compara, para cinco palabras de las preguntas frecuentes, el lema de spaCy con la raíz del `SnowballStemmer("spanish")` de NLTK.

4. Pruebas del preprocesamiento. Al menos dos: `limpiar` deja Q6 sin etiquetas y con `[correo]` en vez del correo, y `limpiar` conserva números y códigos (por ejemplo, "error 500").

5. Búsqueda base. Con `TfidfVectorizer` de scikit-learn, vectoriza las ocho preguntas frecuentes preprocesadas y elige para cada pregunta de prueba la de mayor similitud coseno. Prueba dos variantes: indexar solo la pregunta frecuente, o la pregunta más su respuesta publicada. Repite con `CountVectorizer` para comparar. Entrega una lista de stopwords en español: la opción `stop_words="english"` no sirve aquí.

6. Búsqueda con embeddings. Calcula en una sola llamada los embeddings de las ocho preguntas frecuentes y guárdalos en `datos/faq_embeddings.json`, junto con el nombre del modelo de embeddings: ese archivo es tu vector store, y solo sirve con ese modelo. Por cada pregunta del cliente calcula solo su embedding y compáralo con los guardados por similitud coseno.

7. Comparación y umbral. En una tabla: pregunta, pregunta frecuente esperada, la elegida por TF-IDF, por CountVectorizer y por embeddings, cada una con su similitud, y si acierta. Fíjate en Q1 (la palabra "clave" también aparece en la respuesta de F7) y en Q3 ("safari" solo aparece en la respuesta de F4). Con esos valores, elige el umbral de similitud de

embeddings bajo el cual el asistente deriva y justifícalo: ¿qué similitud obtiene Q5, que comparte la palabra "plan" con F5? Las similitudes de TF-IDF y de embeddings no se comparan entre sí: el umbral vale para el modelo de embeddings que usaste.

Parte 4 · Prompts

En `prompts.md`:

- 1. Qué es tu prompt.** Empieza con dos o tres líneas: qué parte del prompt de tu asistente es fija y qué parte cambia en cada consulta.
- 2. Mensaje de sistema** con rol (asistente de la mesa de ayuda de Nube Sur), contexto (quién lee la respuesta y de dónde sale la información), restricciones (usar solo la respuesta publicada que recibe; no inventar precios, plazos ni funciones; si esa respuesta no contesta la pregunta, derivar; tratar de tú al cliente; máximo 50 palabras) y formato de salida. El **mensaje de usuario** separa los datos con marcas claras: `Pregunta del cliente:`, `Pregunta frecuente (F...):` y `Respuesta publicada:`.
- 3. Salida en JSON** con tres campos: `faq` (el código, por ejemplo "F3", o `null`), `respuesta` (el texto para el cliente) y `derivar` (`true` o `false`). Cuando deriva, `respuesta` es exactamente "Te derivó con un ejecutivo.". Lee la salida con `json.loads` dentro de un `try`: si no es JSON válido o le falta un campo, el asistente deriva y lo registra.
- 4. Zero-shot y few-shot.** Escribe una versión zero-shot (solo instrucciones) y una few-shot con dos ejemplos propios como turnos `user` y `assistant`: uno que se responde con la respuesta publicada y otro que se deriva. Los ejemplos no pueden ser Q1 a Q6. Compara ambas versiones en las seis preguntas de prueba: JSON válido, pregunta frecuente correcta, derivación correcta y tokens de entrada. Agrega dos líneas sobre el prompting instruccional: si una versión con pasos numerados (leer la pregunta, revisar si la respuesta publicada la contesta, redactar o derivar) mejoraría tu asistente, y por qué; si la pruebas, regístrala como una versión más.
- 5. Parámetros.** Elige y justifica en una línea cada uno: `temperature` baja (entre 0 y 0,3), porque la tarea es reformular una respuesta fija; `max_tokens` acotado y con margen sobre el largo pedido, porque es un tope de seguridad y no controla el largo; `top_p` sin tocar, porque ya ajustas `temperature`. Revisa en la documentación oficial si tu modelo acepta `temperature` y si usa `max_tokens` o `max_completion_tokens`.
- 6. Iteraciones.** Al menos tres, cambiando una cosa a la vez y probando siempre con las mismas preguntas. En una tabla: versión, cambio, por qué, efecto (en claridad, estructura o control del comportamiento del modelo) y evidencia (la fila de `resultados.csv`).
- 7. Limitaciones.** Prueba y registra tres: la **alucinación**, enviando Q5 al modelo junto con la respuesta publicada de F5 aunque tu umbral la derive (¿el prompt, por sí solo, evita que invente un precio?); la **sensibilidad al wording**, con tres paráfrasis de Q3 (¿cambian la pregunta frecuente elegida o la respuesta?); y la **falta de control**, con la misma pregunta cinco veces con `temperature` 0,2 y cinco veces con 1,0 (¿cuántas salidas distintas y cuántos JSON inválidos?).
- 8. Un prompt adicional**, de validación o de reformulación. Por ejemplo, uno que reciba la respuesta publicada y la respuesta generada y conteste solo "SI" o "NO" a "¿la respuesta

generada agrega información que no está en la publicada?", o uno que reescriba la respuesta publicada en un tono más cordial sin cambiar ningún dato. Aplícalo a las seis respuestas y registra qué detectó o qué cambió.

Parte 5 · Evaluación y consumo eficiente

- 1. Protocolo.** Escríbelo antes de medir, en el README o en `protocolo.md`: qué preguntas se evalúan (Q1 a Q6), contra qué referencias, con qué tokenizador (uno que no parta ni descarte las palabras con tilde o ñ, por ejemplo quitando las tildes igual en la referencia y en la respuesta; el mismo para ambas), qué métricas se calculan, qué significa cada una y qué valor marca una respuesta como "revisar".
- 2. Métricas automáticas.** Para cada respuesta, ROUGE-L (F1) y BLEU con suavizado (las respuestas son cortas) contra la respuesta de referencia. Si usas `rouge-score`, pásale tu tokenizador. En Q5, lo que cuenta es que el asistente derive y no invente.
- 3. Métricas con pauta.** Valora de 1 a 3 la **coherencia** (3: texto gramatical, sin contradicciones y en el tono pedido) y la **relevancia** (3: responde lo que se preguntó con los datos de la respuesta publicada, sin agregar nada).
- 4. resultados.csv**, una fila por pregunta, estrategia de prompting, versión del prompt y temperature, con las columnas `fecha`, `pregunta`, `faq_esperada`, `faq_elegida`, `similitud`, `derivo`, `estrategia_prompt` (`zero-shot`, `few-shot` o `instruccional`), `version_prompt`, `temperature`, `rougeL_f`, `bleu`, `coherencia_1a3`, `relevancia_1a3`, `tokens_total` y `comentario`.
- 5. consumo.csv**, una fila por llamada a la API, con los datos del campo `usage` de cada respuesta: `fecha`, `pregunta`, `operacion` (`embeddings`, `generar` o `validar`), `modelo`, `prompt_tokens`, `completion_tokens` (0 en los embeddings) y `total_tokens`.
- 6. Consumo eficiente.** Con `consumo.csv`, calcula tres cifras: las llamadas de generación que ahorró el umbral de derivación; los tokens de embeddings que ahorras al guardar los de las preguntas frecuentes en vez de recalcularlos en cada consulta; y cuántos tokens de entrada agrega el few-shot frente al zero-shot. Proyecta las tres a 1.000 consultas; para el umbral, usa el dato del jefe de soporte: 4 de cada 10 consultas tienen respuesta en las preguntas frecuentes y las otras 6 deberían derivarse. Indica también cuántas preguntas derivó tu umbral en Q1 a Q6. Si lo expresas en dinero, usa los precios vigentes que publica el proveedor e indica la fecha en que los consultaste; no los inventes.
- 7. Ajustes.** Identifica en tus números al menos dos ajustes (en la limpieza, el umbral, el prompt o los parámetros), aplícalos uno a la vez y registra el antes y el después.
- 8. Recomendación.** En el informe PDF para el jefe de soporte (ver Entrega), en 5 a 8 líneas: ¿el asistente pasa a un piloto en el chat, pasa con ajustes o no pasa? Apóyala en tus tablas.

Insumos

Todos los datos son ficticios. Los correos usan el dominio `.test`, reservado para pruebas.

Preguntas frecuentes de Nube Sur

#	Pregunta frecuente	Respuesta publicada
F1	¿Cómo recupero mi contraseña?	En la pantalla de ingreso, elige "Olvidé mi contraseña"; te llegará un enlace válido por 30 minutos.
F2	¿Cómo agrego un usuario a mi cuenta?	Un administrador entra a Configuración → Usuarios → Invitar y escribe el correo de la persona.
F3	¿Puedo exportar mis clientes a Excel?	Sí: en Clientes, usa el botón Exportar y elige formato XLSX.
F4	¿Qué navegadores son compatibles?	Las dos últimas versiones de Chrome, Firefox, Edge y Safari.
F5	¿Cómo cambio el plan contratado?	En Facturación → Plan puedes subir de plan en el momento; para bajar, el cambio aplica al mes siguiente.
F6	¿Dónde descargo mis facturas?	En Facturación → Documentos están todas las facturas en PDF.
F7	¿La plataforma tiene API?	Sí, con clave por cuenta; la documentación está en el menú Desarrolladores.
F8	¿Cómo elimino mi cuenta?	Escribe a soporte desde el correo del administrador; la eliminación se confirma en 5 días hábiles.

Preguntas de prueba

#	Pregunta del cliente, tal como llega	Pregunta frecuente esperada	Respuesta de referencia
Q1	olvidé la clave y no puedo entrar!!	F1	Elige "Olvidé mi contraseña" en la pantalla de ingreso y recibirás un enlace válido por 30 minutos.
Q2	Necesito pasar la lista de clientes a una planilla	F3	En Clientes, usa Exportar y elige el formato XLSX.
Q3	¿funciona en safari?	F4	Sí, funciona en las dos últimas versiones de Safari, además de Chrome, Firefox y Edge.
Q4	quiero sumar a mi colega como usuario	F2	Un administrador puede invitarlo desde Configuración → Usuarios → Invitar con su correo.
Q5	¿cuánto cuesta el plan anual?	ninguna	Te derivó con un ejecutivo.

#	Pregunta del cliente, tal como llega	Pregunta frecuente esperada	Respuesta de referencia
Q6	<p>Hola!!! necesito la factura de septiembre y no la encuentro en ningún lado. ¿Me la pueden mandar a contabilidad@losaromos.test?</p>
Enviado desde mi iPhone	F6	Puedes descargar tus facturas en PDF desde Facturación → Documentos.

Q5 no tiene respuesta en las preguntas frecuentes: el asistente debe derivar y no inventar un precio. Q6 trae HTML, ruido y un correo: pone a prueba tu limpieza, y el asistente no debe prometer que enviará nada por correo.

Los mismos datos, listos para copiar

datos/faq.json:

```
[
  {"id": "F1", "pregunta": "¿Cómo recupero mi contraseña?", "respuesta": "En la pantalla de ingreso, elige \"Olvidé mi contraseña\"; te llegará un enlace válido por 30 minutos."},
  {"id": "F2", "pregunta": "¿Cómo agrego un usuario a mi cuenta?", "respuesta": "Un administrador entra a Configuración → Usuarios → Invitar y escribe el correo de la persona."},
  {"id": "F3", "pregunta": "¿Puedo exportar mis clientes a Excel?", "respuesta": "Sí: en Clientes, usa el botón Exportar y elige formato XLSX."},
  {"id": "F4", "pregunta": "¿Qué navegadores son compatibles?", "respuesta": "Las dos últimas versiones de Chrome, Firefox, Edge y Safari."},
  {"id": "F5", "pregunta": "¿Cómo cambio el plan contratado?", "respuesta": "En Facturación → Plan puedes subir de plan en el momento; para bajar, el cambio aplica al mes siguiente."},
  {"id": "F6", "pregunta": "¿Dónde descargo mis facturas?", "respuesta": "En Facturación → Documentos están todas las facturas en PDF."},
  {"id": "F7", "pregunta": "¿La plataforma tiene API?", "respuesta": "Sí, con clave por cuenta; la documentación está en el menú Desarrolladores."},
  {"id": "F8", "pregunta": "¿Cómo elimino mi cuenta?", "respuesta": "Escribe a soporte desde el correo del administrador; la eliminación se confirma en 5 días hábiles."}
]
```

datos/preguntas_prueba.csv:

```
id,pregunta,faq_esperada,referencia
Q1,olvidé la clave y no puedo entrar!!,F1,"Elige ""Olvidé mi contraseña"" en la
pantalla de ingreso y recibirás un enlace válido por 30 minutos."
Q2,Necesito pasar la lista de clientes a una planilla,F3,"En Clientes, usa Exportar
y elige el formato XLSX."
Q3,¿funciona en safari?,F4,"Sí, funciona en las dos últimas versiones de Safari,
además de Chrome, Firefox y Edge."
Q4,quiero sumar a mi colega como usuario,F2,"Un administrador puede invitarlo desde
Configuración → Usuarios → Invitar con su correo."
Q5,¿cuánto cuesta el plan anual?,,Te derivo con un ejecutivo.
Q6,"<p>Hola!!! necesito la factura de septiembre y no la encuentro en ningún lado.
¿Me la pueden mandar a contabilidad@losaromos.test?</p><br/>Enviado desde mi
iPhone",F6,"Puedes descargar tus facturas en PDF desde Facturación → Documentos."
```

`.env.example` (solo los nombres; los valores van en tu `.env`, que nunca se sube):

```
# openai o huggingface
PROVEEDOR_IA=
OPENAI_API_KEY=
HF_TOKEN=
MODELO_CHAT=
MODELO_EMBEDDINGS=
```

Forma de las respuestas, para simular la API en tus pruebas

Respuesta abreviada del endpoint de chat de OpenAI (`POST /v1/chat/completions`). El texto que pediste en JSON llega como una cadena dentro de `content`:

```
{
  "object": "chat.completion",
  "model": "<modelo de chat>",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "{\\"faq\\": \\"F4\\", \\"respuesta\\": \\"Sí, funciona en las dos
últimas versiones de Safari.\\", \\"derivar\\": false}"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {"prompt_tokens": 412, "completion_tokens": 31, "total_tokens": 443}
}
```

Respuesta abreviada del endpoint de embeddings de OpenAI (`POST /v1/embeddings`); los vectores reales tienen cientos o miles de números:

```
{
  "object": "list",
  "data": [
    {"object": "embedding", "index": 0, "embedding": [0.0123, -0.0456, 0.0789]},
    {"object": "embedding", "index": 1, "embedding": [0.0231, 0.0114, -0.0672]}
  ],
  "model": "<modelo de embeddings>",
  "usage": {"prompt_tokens": 14, "total_tokens": 14}
}
```

Si usas Hugging Face, su endpoint de chat compatible con el formato de OpenAI responde con la misma forma; la respuesta de embeddings cambia según la tarea y el proveedor, así que revisala en la documentación oficial antes de escribir tu simulación.

Condiciones

- **Individual.** Puedes conversar ideas con tus compañeros/as, pero el código, los prompts y el informe son tuyos.
- **Datos ficticios.** Nube Sur, sus clientes y sus correos son inventados. Trabaja solo con los insumos de esta actividad y con datos ficticios.
- **Ninguna credencial ni clave** en la entrega: ni en el código, los notebooks, el historial de commits, las capturas ni el informe. Si una clave llega a publicarse, revócala en el panel del proveedor, crea una nueva y anota en el README que lo hiciste, sin la clave.
- **Puedes reutilizar lo que construiste en el módulo** (el cliente de la API, el preprocesamiento, la evaluación y tus prompts), indicando en el README de qué trabajo viene cada pieza y qué cambiaste.
- **Si usas un asistente de IA** para escribir código o texto, decláralo en el README e indica qué revisaste y qué corregiste.
- **Acceso a la API.** Usa el acceso y los modelos que indique tu tutor/a. Las pruebas unitarias deben pasar sin conexión; las métricas y el consumo se calculan con llamadas reales.

Entrega

En la Tarea del LMS entrega tres cosas:

1. **El enlace a tu repositorio** en GitHub, público porque lo enlazarás desde tu portafolio (revisa antes que no tenga claves), con esta estructura mínima:

```

asistente-faq-nube-sur/
├─ README.md           instalación, ejecución, arquitectura, endpoints y qué
falta
├─ requirements.txt
├─ .env.example         nombres de las variables, sin valores
├─ .gitignore           incluye .env
├─ datos/
│   └─ faq.json
│   └─ preguntas_prueba.csv
│   └─ faq_embeddings.json
├─ nubesur_ia/          o cliente_ia.py: el cliente de la API
├─ preprocesar.py
├─ buscar.py            TF-IDF, CountVectorizer y embeddings
├─ asistente.py         el flujo completo: python asistente.py "¿funciona en
safari?"
├─ evaluar.py           métricas y registro
├─ prompts.md
├─ resultados.csv
├─ consumo.csv
└─ tests/

```

2. Un informe breve en PDF (2 a 4 páginas) para el jefe de soporte: el diagrama y las decisiones de arquitectura, la tabla de búsqueda, el resumen de resultados y de consumo, los ajustes y tu recomendación.
3. Una captura de la salida de `pytest`, sin claves ni datos personales visibles.

Cómo se evalúa

La pauta tiene 40 puntos, organizados por los componentes de la competencia del módulo. Cada fila indica qué se revisa y en qué parte de la actividad, y se puntúa así: en las filas de 2 puntos, 2 si cumple todo lo que indica, 1 si cumple al menos la mitad y 0 si cumple menos de la mitad; en las de 3 puntos, 3 si cumple todo, 2 si falta un elemento, 1 si cumple al menos uno y 0 si no cumple ninguno.

Componente de la competencia	Qué se revisa	Puntos
Implementar la arquitectura básica de una aplicación que integre modelos de IA		8
	IA generativa frente a reglas, con Q1 y Q2 y una tarea que conviene resolver con una regla; tipos de IA generativa y familias de modelos usados y descartados; casos de uso cubiertos y uno por agregar (Parte 1, pasos 1 a 3)	3
	Diagrama con los cinco componentes, la función de cada uno y el flujo de extremo a extremo numerado; qué parte es solicitud–respuesta y cuál pipeline (Parte 1, pasos 4 y 5)	3
	Tecnologías elegidas y descartadas con su motivo; justificación con escalabilidad, seguridad y mantenimiento; la clave vive solo en el servidor (Parte 1, pasos 6 y 7)	2

Componente de la competencia	Qué se revisa	Puntos
Mediante APIs		8
	GET para verificar y POST para embeddings y generación, con cuerpo JSON y lectura de la respuesta después de revisar el código de estado; clave y modelos por variables de entorno; tiempo de espera explícito (Parte 2, pasos 1 a 4)	2
	Manejo distinto de 401, 429, 5xx y tiempo agotado; docstrings; cinco o más pruebas que pasan sin conexión ni clave; tabla de endpoints con enlaces a la documentación oficial (Parte 2, pasos 5 a 8)	3
	Los mensajes que el cliente envía al endpoint de chat: mensaje de sistema con rol, contexto y restricciones; mensaje de usuario con los datos delimitados; salida JSON de la respuesta validada con <code>json.loads</code> ; comparación de zero-shot y few-shot con dos ejemplos propios (Parte 4, pasos 1 a 4)	3
Preprocesamiento de texto para consumo por modelos de lenguaje		8
	Funciones reutilizables por etapa, con sus pruebas; Q6 queda sin HTML, sin ruido y con <code>[correo]</code> ; el texto que va al modelo conserva su forma natural (Parte 3, pasos 1, 2 y 4)	3
	Tokens, lemas, stopwords (con la decisión sobre "no") y bigramas de Q6; lema frente a raíz (Parte 3, paso 3)	2
	Búsqueda con TF-IDF, CountVectorizer y embeddings comparada en una tabla; umbral de derivación justificado con las similitudes (Parte 3, pasos 5 a 7)	3
Garantizando el consumo eficiente		6
	<code>consumo.csv</code> con los tokens de cada llamada tomados de <code>usage</code> (Parte 5, paso 5)	2
	Parámetros justificados y verificados para el modelo usado; embeddings de las preguntas frecuentes calculados en una llamada y guardados; reintentos limitados (Parte 4, paso 5; Parte 3, paso 6; Parte 2, paso 5)	2
	Ahorro del umbral y del almacenamiento de embeddings, y costo del few-shot, proyectados a 1.000 consultas (Parte 5, paso 6)	2
La evaluación de resultados		6
	Protocolo escrito antes de medir; ROUGE-L y BLEU bien calculados con un tokenizador que no parte las palabras con tilde o ñ; coherencia y relevancia con pauta; <code>resultados.csv</code> completo (Parte 5, pasos 1 a 4)	2

Componente de la competencia	Qué se revisa	Puntos
	Tres o más iteraciones registradas con su efecto; pruebas de alucinación, wording y falta de control; prompt de validación o de reformulación aplicado (Parte 4, pasos 6 a 8)	2
	Dos o más ajustes identificados en los números, con su antes y después; recomendación apoyada en las tablas, en el informe PDF que resume diagrama, búsqueda, resultados y consumo (Parte 5, pasos 7 y 8; Entrega)	2
En un entorno de desarrollo de software		4
	Repositorio ordenado: README que permite instalar y ejecutar, <code>requirements.txt</code> , estructura de carpetas y <code>pytest</code> que pasa en un entorno limpio (Entrega y Condiciones)	2
	<code>.gitignore</code> con <code>.env</code> , <code>.env.example</code> sin valores y ninguna clave en el código, las capturas ni el historial; commits que muestran el avance (Entrega y Condiciones)	2
Total		40

Una clave publicada en el repositorio o en su historial deja en 0 la fila «`.gitignore` con `.env`, `.env.example` sin valores y ninguna clave...» (2 puntos del componente «En un entorno de desarrollo de software»), y tu tutor/a te pedirá revocarla antes de la coevaluación.

Nota (exigencia 60 %): si tu puntaje p es 24 o más, $\text{nota} = 4,0 + 3 \times (p - 24) / 16$; si es menor, $\text{nota} = 1,0 + 3 \times p / 24$. Se redondea a un decimal. Por ejemplo, 40 puntos dan un 7,0; 32 puntos, un 5,5; 24 puntos, un 4,0, y 20 puntos, un 3,5.

Contenidos del plan que integra esta actividad

La actividad integra los 39 temas de los cuatro contenidos del plan del módulo. La tabla muestra en qué parte se trabaja cada uno.

Parte de la actividad	Contenidos del plan
Parte 1, pasos 1 y 3: por qué IA generativa y qué casos de uso	Conceptos generales de IA generativa: qué es la IA generativa y para qué sirve. Diferencias entre IA generativa y IA tradicional basada en reglas. Casos de uso: generación de texto, generación de código, síntesis de imágenes, QA, summarization y generación de tests.
Parte 1, paso 2: tipos de IA y de modelos	Tipos de IA generativa: texto, imagen, audio y video. Tipos de modelos: autorregresivos, de difusión y basados en Transformers.
Parte 1, pasos 4 y 5: diagrama funcional y arquitectura	Componentes: cliente, servidor API, modelo de IA, módulo de preprocesamiento y base de datos/vector store. Arquitecturas: request-response y pipelines.

Parte de la actividad	Contenidos del plan
Parte 1, pasos 4, 6 y 7: flujo de datos, tecnologías y justificación	Ecosistema tecnológico de la IA. Flujo de datos end-to-end: tecnologías y librerías: OpenAI API, Hugging Face Transformers, TensorFlow, PyTorch, LangChain y Diffusers.
Parte 2, pasos 1 y 2: el cliente y sus tres operaciones	Principios REST y métodos HTTP: GET y POST. Endpoints de generación y embeddings. Ejemplos de uso: parámetros, cuerpos JSON y parsing de respuesta. Modularización de funciones o clases para consumo de APIs.
Parte 2, pasos 2 a 4: librería, configuración, clave y tiempo de espera	Gestión de autenticación y variables de entorno. Librerías: requests y httpx.
Parte 2, paso 5: errores del servicio	Manejo básico de errores y respuestas del servicio.
Parte 2, pasos 6 y 7: docstrings y pruebas	Documentación con docstrings. Pruebas unitarias simples para funciones de consumo de APIs.
Parte 2, paso 8: tabla de endpoints	Documentación oficial: enlaces y organización de los manuales de OpenAI y Hugging Face inference API.
Parte 3, pasos 1, 2 y 4: funciones del pipeline y sus pruebas	Técnicas de limpieza: eliminación de caracteres especiales, html y ruido. Normalización de mayúsculas, minúsculas y estandarización de texto. Tokenización y normalización con librerías como spaCy o NLTK. Estructura modular de un pipeline de preprocesamiento. Funciones reutilizables para cada etapa.
Parte 3, paso 3: tokens, lemas, stopwords y n-gramas de Q6	Conceptos básicos de NLP: token, palabra, lema, stopwords y n-gramas. Lematización y stemming.
Parte 3, pasos 5 a 7: búsqueda y umbral	Vectorización básica: CountVectorizer y TF-IDF.
Parte 4, pasos 1 a 3: el prompt, sus mensajes y la salida en JSON	Qué es un prompt. Estructura básica de entrada para modelos de lenguaje. Prácticas recomendadas: claridad, delimitación del rol, contexto, restricciones y evitar ambigüedad. Formateo de salidas: listas, JSON y estructura tabular.
Parte 4, paso 4: zero-shot y few-shot	Tipos de prompting: zero-shot, few-shot e instructional prompting.
Parte 4, paso 5: parámetros	Prompt y parámetros: cómo influyen temperature, max_tokens y top_p.
Parte 4, pasos 6 y 7: iteraciones y limitaciones	Iteración y ajuste de prompts según resultados obtenidos. Limitaciones: alucinaciones, falta de control y sensibilidad al wording.
Parte 4, paso 8: prompt de validación o de reformulación	Ejemplos prácticos: prompt para generar código, resumen, validación y reformulación.

Parte de la actividad	Contenidos del plan
Parte 5, pasos 1 a 4: protocolo, métricas y <code>resultados.csv</code>	Métricas básicas de calidad de respuestas: coherencia, relevancia, BLEU y ROUGE. Registro de resultados de evaluación.
Parte 5, pasos 5 a 8: consumo, ajustes y recomendación	Identificación de ajustes iniciales a partir de métricas obtenidas.

Después de entregar

- **Coevaluación por pares.** Tu tutor/a te asigna un compañero/a: clonas su repositorio, ejecutas sus pruebas y revisas su trabajo con la tabla de la Parte A de la coevaluación. Él o ella hace lo mismo con el tuyo.
- **Autoevaluación del módulo 2.** Se completa en dos momentos. Al entregar, en su Parte 2 revisas tu actividad frente a los mismos componentes de la competencia de esta pauta y estimas tu puntaje (también completas las Partes 1, 3 y 4). Al recibir la devolución, anotas su puntaje en la Parte 2 y completas la Parte 5, antes de publicar el portafolio.
- **Portafolio.** El asistente va en el elemento de integración de tu portafolio, con el enlace al repositorio, el diagrama, la tabla de resultados y tu recomendación.
- **Devolución del tutor/a.** Recibes la pauta con el puntaje de cada fila y un comentario con una fortaleza, una mejora y un siguiente paso.